Findora

Conversational shop assistant for smartphones & headphones

A multi-turn LLM recommender, re-architected from a structured pipeline into a hybrid tool-calling agent and proven on an evaluation gate.

Group 26

Applied Generative AI · SS 2026

Arthur Galois · Jakob Meltzer · Sofie Parkesaas · Muhammad Azeem Shahzad

Abstract

Findora is a conversational recommender that acts as a shop assistant for tech products, taken from prototype to a deployable design. v1 was a structured LangGraph pipeline: split NLU (router plus slot-filling) guarded by schema validation and entity resolution, vague language ('cheap', 'good camera') grounded in dataset percentiles, hybrid pandas plus dense-embedding retrieval, and TOPSIS ranking. Adding intents one by one proved brittle (whack-a-mole), so we built an evaluation harness (persona simulation, LLM-judge, adversarial / safety suites) and re-architected the dialogue brain into a hybrid tool-calling AGENT over the same deterministic core. On the shared gate the agent roughly doubles conversation quality, reaches 100% on adversarial / safety, and fixes count / compound requests, with no per-scenario code. The pipeline is kept as a fallback.

Initial Situation & Problem

Traditional recommenders work silently (ranked lists, 'people also bought'). Users often start vague and refine as they talk. A conversational recommender (CRS) must instead elicit preferences in real time and adapt.

Core challenges:

- Robust intent detection and preference elicitation from messy natural language.
- Reasoning over a dialogue state to pick the next best action, not blindly answering.
- Safely translating critiques ('cheaper but better camera') into database queries.
- What information does the system actually need to reliably recommend a set of items?

Sample Use Cases

1 · Specific lookup

'A Samsung Android phone with 8GB RAM'

Pre-fills brand, OS and spec, then recommends immediately, ranked, with no needless questions.

2 · Guided exploration

'I want a smartphone'

Asks a focused sequence: OS -> price -> battery -> camera -> RAM, honouring 'any' / 'skip'.

3 · Refinement / critique

'cheaper ones' · 'bigger battery'

Anchors relative terms to the last results; refinements are additive; 'forget X' removes a filter.

4 · Vague & vibe language

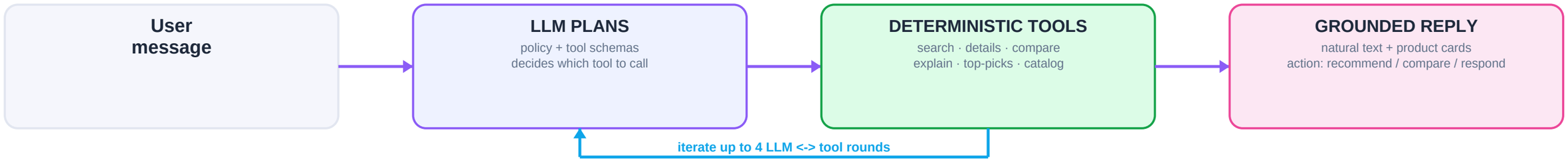
'cheap phone for gaming'

Maps 'cheap' to the p25 price; reads 'gaming' via semantic embeddings over spec descriptions.

Approach: From Pipeline to Tool-Calling Agent

v2 is a hybrid tool-calling agent over a deterministic core (the v1 pipeline, kept as the ENGINE=pipeline fallback).

v2 · Agent loop (default engine)



Grounding lives in the tools: they validate every filter, return only real catalog rows, and honour an explicit count. The LLM only plans and composes (no fabricated specs).

v1 · Deterministic core (the agent calls these as tools; also runs standalone as the fallback)

MEMORY
per-user profiles · 'use my usual' · personalised greeting

RANKING
TOPSIS (MCDM) <-> learned LTR · semantic blend · feedback

OBSERVABILITY
LangSmith tracing · per-turn JSONL · tokens & latency

IMAGES
offline enrichment -> GSMArena / Wikipedia -> card image

Key Design Decisions

- Plan-and-call agent instead of enumerating intents: open-ended and compound requests compose from tools, with no per-scenario code.
- Validate every LLM-proposed slot against a schema; drop unknown / out-of-range / wrong-type, so bad arguments never reach the query engine.
- Grounding lives in deterministic tools (real rows only, explicit counts, honest 'not in catalog').
- Entity resolution as a real layer (alias dictionary plus fuzzy match), not ever-growing prompt rules.
- Deterministic guardrails for the precise bits: undo ('ignore that') and brand exclusion.
- Hybrid retrieval: exact pandas filters plus dense embeddings (model2vec) for 'vibe' queries.
- Transparent TOPSIS ranking; a pluggable learned ranker trains from click feedback.
- Prompt caching on the static system-plus-tools prefix keeps cost and latency down.

Models & Tools

Engine	tool-calling agent (default) <-> LangGraph pipeline (fallback)
Tools	search · details · compare · explain · top-picks · catalog
LLM	OpenRouter · GPT-4o-mini (function-calling) · local Ollama opt.
Embeddings	model2vec · potion-base-8M (static, no GPU)
Ranking	TOPSIS (Hwang & Yoon) · numpy logistic LTR
Data / UI	pandas · 2 x 500 catalog · Streamlit chat plus cards
Observability	LangSmith · per-turn JSONL · tokens & latency
Evaluation	persona sim plus LLM-judge plus 28 adversarial / safety checks

Cross-cutting: skip · undo · multi-intent · category-switch · percentile grounding · memory.

Results, Evaluation & Learnings

0.75
was 0.41
Conversation quality (LLM-judge)

100%
was 82%
Adversarial & safety pass-rate

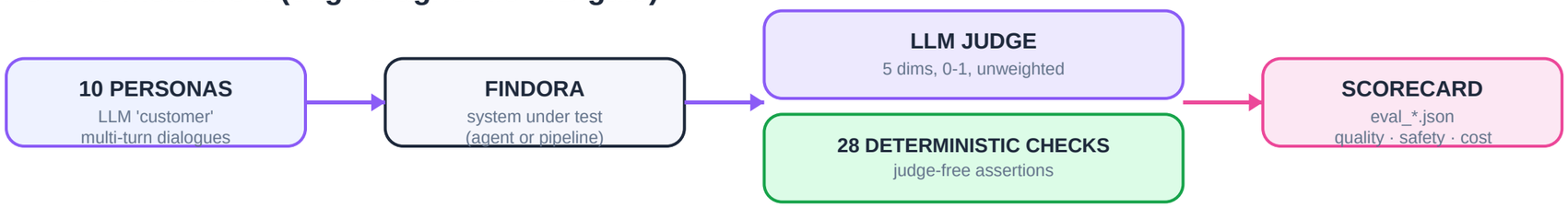
6/6
was 1/6
Count & compound requests

Latency & cost per turn (report the distribution, not just the average)

agent: avg 4.41s · p95 9.64s · max 54.04s | pipeline: avg 7.21s · p95 6.12s · max 221.9s | 56% input tokens cached

The agent wins on PREDICTABILITY and COST, not raw speed: its p95 is higher on genuinely multi-step turns, while the pipeline average is inflated by API-retry outliers (max ~222s).

How we measure it (engine-agnostic A/B gate)



Demo: example conversation

You
suggest me 2 best phones

Findora
Two top picks by value: the Redmi Note 12 Pro+ and the Explorer. (count = 2 honoured)

You
compare them

Findora
Side-by-side: the Pro+ leads on RAM and battery; the Explorer is cheaper. Cards below.

You
compare it with the iPhone 15 Pro Max

Findora
I do not carry the iPhone 15 Pro Max; the closest I have is the iPhone 15 Pro. (no fabrication)

Evaluation Methodology

- Quality = mean of 5 judge dims (goal, honored, grounded, helpful, safe), each 0-1, UNWEIGHTED, averaged over 10 personas.
- Judge-free corroboration: 28 deterministic assertions move 82% -> 100% (count / compound 1/6 -> 6/6) with no LLM judging, so the gain is not a judge artifact.
- Bias guard: --judge-model grades with a different model family than the system under test.
- Variance: persona quality is 0.746 +/- 0.024 over 3 runs (--runs N).
- Reproducible: python eval_harness.py vs ENGINE=pipeline python eval_harness.py.

Learnings & Limitations

What worked

- Build the eval gate FIRST: persona sim plus LLM-judge turned 'feels better' into a defensible number.
- The tool-calling agent fixed open-world gaps (counts, compound, anaphora) with NO per-scenario code.
- Grounding in deterministic tools means the LLM only plans: honest replies, no fabricated specs.
- Hybrid wins: deterministic guardrails plus an LLM brain beat either alone.

What was hard / we would change

- Enumerating intents was whack-a-mole: fixing one scenario broke another. The agent removed that ceiling.
- Optimising latency over-tightened replies and dropped quality; the gate caught it and we re-balanced.
- Agent is cloud-first: function-calling on small local models is unreliable (a known trade-off).
- A few personas are capped by the dataset (obscure products), not the system; richer data would lift them.